

Polynomial Theory of Complex Systems

2026

Cristea Mihnea

mihnea-ioan.cristea@s.unibuc.ro

Ticu Tudor

tudor.ticu@s.unibuc.ro

Abstract

This project investigates the use of the Group Method of Data Handling (GMDH) for short and medium term traffic forecasting. Using historical traffic measurements, we construct learning models that predict average vehicle speed several hours into the future. The GMDH algorithm incrementally builds polynomial regression neurons and selects the best performing components based on validation error, enabling it to capture nonlinear relationships in traffic dynamics. A temporal train/validation split is applied to prevent data leakage, and multiple lagged features are incorporated to model traffic inertia. GMDH provides superior accuracy for longer horizons, where nonlinear effects become more pronounced. These findings highlight the potential of GMDH as a flexible and effective approach for real world traffic prediction.

1 Introduction

The aim of this project is to design and evaluate a machine learning model capable of forecasting traffic data some hours into the future, by training it on historical data. To that end, we used the Group Method of Data Handling a self-organizing modeling technique that incrementally constructs polynomial regression models and selects the best performing components based on validation error. A temporal train-validation split is applied, and lagged traffic features are incorporated to capture traffic inertia. Model performance is assessed using standard regression metrics, including Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), and the coefficient of determination (R^2).

We chose to approach this project both due to it's practical relevance of real-world traffic prediction and to the fact that we found the paper detailing Ivakhnenko's research very interesting and wanted to get a better grasp of the GMDH technique.

The use of GMDH for forecasting real-world behaviours of complex systems has been explored in several previous studies. One notable example is the work of M.C.Acock and Ya.A.Pachepsky, Estimating Missing Weather Data for Agricultural Simulations Using the Group Method of Data Handling (Journal of Applied Meteorology, 1988–2005), which applies GMDH to reconstruct missing weather variables in nonlinear environmental datasets.

Contributions:

- obtaining data - joint effort
- GMDH skeleton - Tudor
- GMDH tuning and reviewing - both
- MLR - Mihnea
- writing report - both

What we learned:

- Tudor: During this project, I gained a better understanding of time series forecasting, the GMDH approach, the way traffic data is recorded, stored and accessed. I also learned about the importance and risks of data leakage in machine learning models (on a first-hand basis). In the future, I would like to learn about more applications of GMDH in the real world, as well as about how other traffic forecasting systems work.
- Mihnea: During this project, I developed a much clearer understanding of the practical differences between linear and nonlinear forecasting models, especially in how they behave under different prediction horizons. Working with both MLR and GMDH helped me see when a simple linear relationship is sufficient and when nonlinear interactions become essential for capturing the true dynam-

ics of traffic flow. In the future, I am interested in learning how external factors like incidents, weather, or road closures can be integrated into forecasting systems to improve real-world accuracy.

2 Approach

All code and datasets used in this project are available at: <https://github.com/mhncrst/BucharestTrafficGMDH/tree/main>

The repository contains:

/flattened-data.csv/ – the processed dataset
/gmdh.py/ – implementation of the GMDH model, dataset builder, and evaluation scripts
/MLR.py/ – implementation of the MLR model

We implemented and evaluated the method in Python 3. The main libraries used were:

- NumPy for matrix computations and least-squares fitting
- Pandas for data loading, cleaning, grouping, shifting targets, and feature engineering
- Scikit-learn for feature standardization (StandardScaler) and evaluation metrics (MAE, RMSE, R^2)

The GMDH works as follows:

- choose the original features to be used as “signals”:
- generate many small models (“neurons”), each using two signals and learning a quadratic polynomial:

$$\hat{y} = a_0 + a_1x_1 + a_2x_2 + a_3x_1^2 + a_4x_2^2 + a_5x_1x_2$$

- fit each neuron on training data using least squares (closed-form linear regression in polynomial features):

$$\hat{\mathbf{a}} = \arg \min_{\mathbf{a}} \|\mathbf{X}\mathbf{a} - \mathbf{y}\|^2$$

- evaluate each neuron on validation data and keep only the best K neurons (lowest validation RMSE):

$$\text{RMSE} = \sqrt{\frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2}$$

- treat outputs of selected neurons as new signals and repeat the process for multiple layers

- stop when validation performance no longer improves (early stopping via patience)

Training and processing both took a relative short amount of time, never more than 20 seconds, even with a lot of layers.

To contextualize the performance of GMDH, we also trained a Multiple Linear Regression (MLR) model using the same features and temporal setup. MLR serves as a strong linear baseline, allowing us to distinguish improvements that come specifically from modeling nonlinear relationships.

To improve results, we added lag features to the starting feature list, which are values of the same type as the ones in the original list, only recorded an hour earlier. This helps the system understand the correlation between the passage of time and the traffic flow, and it leads to better results, especially in longer term forecasts (1hr vs 5hrs).

As we increase the forecast horizon, the prediction task becomes more nonlinear and inherently noisier: the influence of past conditions weakens, interactions between variables become more complex, and simple linear trends no longer hold. In this regime, GMDH begins to outperform linear models because its layered polynomial structure can absorb noise, model higher-order relationships, and adapt to patterns that drift over time.

The effect of forecast horizon on model performance is clearly visible in the experimental results:

• 1 hour ahead:

– **MLR:** MAE = 3.029, RMSE = 4.715, R^2 = 84.61%

– **GMDH:** MAE = 3.300, RMSE = 5.220, R^2 = 81.14%

• 3 hours ahead:

– **MLR:** MAE = 4.153, RMSE = 6.339, R^2 = 72.20%

– **GMDH:** MAE = 4.245, RMSE = 6.422, R^2 = 71.46%

• 6 hours ahead:

– **MLR:** MAE = 4.726, RMSE = 6.929, R^2 = 66.75%

– **GMDH:** MAE = 4.505, RMSE = 6.581, R^2 = 70.01%

3 Limitations

The main limitation we encountered and had to manage while working on this project was the scarcity of historic traffic data available to the public. Even after we obtained a significant amount of data, it was all recorded in the same month(August 2024), so we were unable to explore the effects that seasons/the school year/weather etc. have on traffic.

There was also the limitation of only being allowed to request 20 reports, so the data we used was hourly, which prevented us from being able to process short term traffic fluctuations.

While the feature list we used and the amount of data meant that the GMDH was in no way demanding of processing power, we believe that due to the fact that neurons are created combinatorially, increases in feature list size would extend into potential concerns, so understanding if the model is effective in a city-wide system would require further investigation.

A further limitation of our approach is that the dataset does not include information about unexpected traffic disruptions such as car crashes, road closures, or construction events. These incidents can cause abrupt changes that are not captured by lag features alone. Incorporating an incident-related variable—either from an external traffic API could significantly improve the model’s ability to handle sudden, nonlinear shifts in traffic flow.

4 Conclusions and Future Work

While doing this project, we learned a lot of things that we otherwise might have never found out about and we also had to use problem solving skills to manage and work around the limitations and issues that came up. The mix of challenge and learning proved to be quite enjoyable.

The project can definitely be improved, but we believe that is more a matter of getting better data sources(which is hard because most are paywalled) than actual code/system issues. A solution to this issue could be creating our own dataset: making API calls to a live-traffic API and storing the data over a long period of time.

As for what we could have done differently, the only thing that comes to mind is organisation.

References

- A. G. Ivakhnenko. 1971. [Polynomial theory of complex systems](#). *IEEE Transactions on Systems, Man, and Cybernetics*, SMC-1(4):364–378.